DO SIMULADOR AO SILÍCIO · ENCONTRO 6 · LIGA PROTOFPGA

Gabarito

Parte A — Atividade em aula

Item 1 — as instruções do `aula4_laco.asm` que não existem no MIPS_S:

Instrução	Quantas vezes	O que pôr no lugar
`add`	1	`addu`
`addi`	2	`addiu`
`syscall`	5	Os quatro de impressão viram um `sw` da soma em `0xFFFF0008`; o do serviço 10 vira `fim: j fim`

Item	Resposta
2	`li $s1, 8` virou `addiu $17, $0, 8`; `move $t2, $t4` virou `addu $10, $0, $12`. As duas existem no MIPS_S. O mesmo vale para `la $s0, vetor`, que virou `lui` e `ori`.
3	a `0000000000000101` (5) · b `0000000000001010` (10) · c `0000000000001011` (11) · d `0000000000001010` (10) · e `0000000000001001` (9) · f `0000000000000100` (4) · g `0000000000000000` (0) · h `1111111111111111` (65535).
4	Saída conferida no MARS com a entrada 1, 5, 4, 2, 16, 16, 8, 1, 0, 16, 0: os mesmos valores do item 3. Os erros mais comuns são esquecer que BTNR desloca para a direita (9 ÷ 2 = 4, e não 4,5) e que BTNC substitui o contador, em vez de somar.
5	0 − 1 = −1, que em complemento de dois é `0xFFFFFFFF`: todos os bits em 1. O `andi $s0, $s0, 0xFFFF` mantém os 16 de baixo, e os 16 LEDs acendem. Com `addiu` não há exceção. Apertando BTNU em seguida, `0xFFFF` + 1 = `0x10000`; o `andi` corta o bit 16, e os LEDs apagam todos: o contador "dá a volta", como um odômetro.
6	`sll $t1, $s0, 1` (o dobro) e `addu $s0, $s0, $t1` (x + 2x). Também vale `addu $t1, $s0, $s0` no lugar do `sll`.
7	**15** e **45** (5 × 3 e 15 × 3). Conferido no MARS.
8	Daria: `addiu $t1, $zero, 3`, `multu $s0, $t1` e `mflo $s0`. Mas são 4 + 67 + 4 = **75 ciclos**, contra 4 + 4 = **8 ciclos** do `sll` com `addu`. Multiplicar por uma constante pequena com deslocamentos e somas é o que um compilador faz.
9	Mudanças: `add` → `addu`; as duas `addi` → `addiu`; os blocos de impressão saem e entram `lui $s7, 0xFFFF` e `sw $t1, 8($s7)`; o serviço 10 vira `fim: j fim`. As linhas `.data` com `msg1` e `msg2` podem sair. Ver o programa abaixo.
10	`0x00000029` = 41, a soma do vetor.
11	LED5, LED3 e LED0: 41 = 32 + 8 + 1 = `0000000000101001`.

Item 9 — uma resposta possível, conferida no MARS (só instruções do MIPS_S):

```
        .data
vetor:  .word  7, 3, 9, 2, 5, 8, 1, 6
        .text
        .globl main
main:   la    $s0, vetor
        li    $s1, 8
        li    $t0, 0
        li    $t1, 0
        lw    $t2, 0($s0)
laco:   slt   $t3, $t0, $s1
        beq   $t3, $zero, fim
        lw    $t4, 0($s0)
        addu  $t1, $t1, $t4     # era add
        slt   $t3, $t2, $t4
        beq   $t3, $zero, pula
        move  $t2, $t4
pula:   addiu $t0, $t0, 1        # era addi
        addiu $s0, $s0, 4        # era addi
        j     laco
fim:    lui   $s7, 0xFFFF
        sw    $t1, 8($s7)    # LEDs = soma = 41
para:   j     para
```

O `aula6_soma_leds.asm`, distribuído com o material, é a versão completa: guarda a soma em `$s2` e o maior em `$s3` e alterna os dois nos LEDs a cada botão apertado.

Sobre os resultados deste gabarito

Todos os programas foram montados e executados no MARS 4.5, e a lista de instruções de cada um foi conferida contra as 37 do MIPS_S. No MARS, a escrita em `0xFFFF0008` aparece como memória comum (a região MMIO); na placa, é ela que acende os LEDs.

Parte B — Desafio

Uma solução possível: o `desafio_calculadora_mars.asm`, distribuído com o material. O laço principal (as três funções de E/S são as do `aula6_contador_mars.asm`):

```
laco:   jal   espera_botao
        addu  $s1, $v0, $zero   # botão: precisa sobreviver ao próximo jal
        jal   le_chaves
        addu  $s2, $v0, $zero   # chaves
        andi  $t0, $s1, 0x01    # BTNC: carrega
        beq   $t0, $zero, nao_c
        addu  $s0, $s2, $zero
nao_c:  andi  $t0, $s1, 0x02    # BTNU: soma
        beq   $t0, $zero, nao_u
        addu  $s0, $s0, $s2
nao_u:  andi  $t0, $s1, 0x10    # BTND: subtrai
        beq   $t0, $zero, nao_d
        subu  $s0, $s0, $s2
nao_d:  andi  $t0, $s1, 0x04    # BTNL: multiplica
        beq   $t0, $zero, nao_l
        multu $s0, $s2
        mflo  $s0               # 32 bits de baixo do produto
nao_l:  andi  $t0, $s1, 0x08    # BTNR: resto
        beq   $t0, $zero, nao_r
        beq   $s2, $zero, nao_r   # dividir por 0: nada
        divu  $s0, $s2
        mfhi  $s0               # o resto fica em hi
nao_r:  andi  $s0, $s0, 0xFFFF
        addu  $a0, $s0, $zero
        jal   mostra_leds
        j     laco
```

Conferido no MARS: C 100 → `0000000001100100` (100); U 23 → 123; L 2 → 246; R 7 → 1; D 2 → `1111111111111111` (65535). Com as chaves em 0, BTNR não muda nada.

Pergunta extra. O botão foi guardado em `$s1`. Ele chega em `$v0`, mas `le_chaves` também devolve em `$v0` e apagaria o código. E não pode ir para um `$t`: pela convenção do encontro 5, os `$t` podem mudar em qualquer `jal` (na versão MARS, `mostra_leds` usa `$t1`, `$t2` e `$t3`). Os `$s` atravessam as chamadas.

Parte C — Exercícios de revisão

1. Não: as duas dão exatamente o mesmo resultado de 32 bits, com ou sem sinal. A diferença é que `add` interrompe o programa com uma exceção quando a soma transborda, e `addu` não. Ao trocar, perde-se esse aviso. O MIPS_S não precisa de `add` porque não tem um mecanismo de exceções: sem ele, `add` seria idêntica a `addu`.

2. (a) `addiu $t0, $t0, -1`; (b) `subu $t2, $t0, $t1`; (c) `multu $t1, $t2` seguida de `mflo $t3`.

3. Podem: `li $t0, 5` (vira `addiu $8, $0, 5`), `move $t1, $t0` (vira `addu $9, $0, $8`) e `blt $t0, $t1, L` (vira `slt $1, $8, $9` e `bne $1, $0, L`). Não podem: `neg $t2, $t0` (vira **`sub`** `$10, $0, $8`) e `subi $t3, $t0, 1` (vira **`addi`** e **`sub`**). Conferido na coluna Basic do MARS.

4. (a) `blez` 4 + `lw` 5 + `addu` 4 + `sw` 4 + `addiu` 4 + `addiu` 4 + `j` 4 = **29 ciclos** por volta. (b) 34 + 11 × 29 + 8 = 34 + 319 + 8 = **361 ciclos**. (c) 361 × 20 ns = **7,22 µs** — o mesmo resultado da apresentação do MIPS_S.

5. 67 × 20 ns = 1 340 ns = **1,34 µs**. Nesse tempo cabem 67 ÷ 4 = 16 instruções `addu` completas (16,75).

6. Os LEDs 15 e 0 são os bits 15 e 0: `1000 0000 0000 0001` = `0x8001`:

```
lui  $s7, 0xFFFF
ori  $t0, $zero, 0x8001
sw   $t0, 8($s7)
```

7. Quatro instruções (e só três no laço):

```
main:  lui  $s7, 0xFFFF
laco:  lw   $t0, 4($s7)    # lê as chaves
       sw   $t0, 8($s7)    # acende os LEDs
       j    laco
```

> Cada volta leva 5 + 4 + 4 = 13 ciclos, ou 260 ns: a cópia é atualizada quase 4 milhões de vezes por segundo, e o LED parece seguir a chave instantaneamente.

8. `sw` não é uma instrução especial: é a mesma escrita na memória do encontro 3. O processador nem sabe que existe um LED. Quem decide que um `sw` em `0xFFFF0008` acende LEDs é o periférico — o circuito ligado ao barramento, que reconhece esse endereço e guarda o valor num registrador ligado aos LEDs. É a entrada e saída mapeada em memória.

9. Cinco mudanças: (1) os cinco `syscall` saem — os quatro de impressão viram um `sw` do resultado em `0xFFFF0008`, e o do serviço 10 vira `fim: j fim`; (2) as três `addi` viram `addiu`; (3) `mul $v0, $a0, $v0` vira `multu $a0, $v0` e `mflo $v0`; (4) o programa começa com `li $sp, 0x10014000`, porque usa a pilha e o `$sp` do MARS aponta para uma memória que não existe na placa; (5) o `.data` com as mensagens pode sair. Com 120 = 64 + 32 + 16 + 8 = `0000000001111000`, acendem **LED6, LED5, LED4 e LED3**. Uma resposta, conferida no MARS:

```
main:   li    $sp, 0x10014000  # fim da memória de dados
        li    $s1, 5
        move  $a0, $s1
        jal   fact
        lui   $s7, 0xFFFF
        sw    $v0, 8($s7)      # LEDs = 120
fim:    j     fim
fact:   addiu $sp, $sp, -8
        sw    $ra, 4($sp)
        sw    $a0, 0($sp)
        bne   $a0, $zero, geral
        li    $v0, 1
        j     sai
geral:  addiu $a0, $a0, -1
        jal   fact
        lw    $a0, 0($sp)
        multu $a0, $v0
        mflo  $v0
sai:    lw    $ra, 4($sp)
        addiu $sp, $sp, 8
        jr    $ra
```

10. O serviço 10 é um pedido ao sistema operacional para encerrar o programa, e na placa não há sistema operacional para atendê-lo (nem instrução `syscall`). Depois de `fim: j fim`, o processador continua trabalhando: executa o mesmo `j` para sempre, 12,5 milhões de vezes por segundo. Os LEDs mantêm o último valor escrito, porque o registrador do periférico só muda num novo `sw`. Para recomeçar, aperta-se o CPU_RESET.

11. 100 ms × 50 000 000 ciclos por segundo = 5 000 000 ciclos; ÷ 8 ciclos por volta = **625 000 voltas** = `0x98968`. Não cabe em 16 bits (o máximo é `0xFFFF` = 65 535), então são duas instruções, como no encontro 2:

```
lui  $t1, 0x0009          # $t1 = 0x00090000
ori  $t1, $t1, 0x8968     # $t1 = 0x00098968 = 625 000
```

Conferido no MARS: `$t1` = `0x00098968`. A pseudoinstrução `li $t1, 625000` faz o mesmo, em duas instruções que passam pelo `$at`.